Data Management VO

05 Relational Algebra and Query Processing

Lucas Iacono

Know-Center & Graz University of Technology, Austria

Institute of Human-Centred Computing

Course calendar

- **Part 1: Data Modeling and relational databases**
  - **(5) Mon. 20.04.2026: Relational algebra and query processing**
  - (6) Mon. 27.04.2026: Physical Design and Tuning
  - **Mon. 04.05.2026: Exam #2 no lecture**

Last lecture

- **Query languages (SQL)**
  - **Exercise 2**

- **SQL**
  - **Data Definition Language (DDL)** → Manipulate the database schema
  - **Data Manipulation Language (DML)** → Update and query database
  - **Data Control Language (DCL)** → Modify permissions
  -

- **Questions or comments?**
  - Please also see Discord
  - → Submission of exercise 2: 8th of May

This week: Agenda

- **Relational Algebra**
  - Theoretical foundation for query languages on relational databases
  - Uses algebra for modeling relational data and defining queries on it

- **Query Processing**
  - How can we optimize queries with rewrites?
  - How are queries internally processed?
  - How can queries be analyzed?

Recap: Relation

- Represents a **table**

(precisely a part of a table but we use it as table)

- **Attributes** represent the columns headers
- **Tuples** represent the rows

- **Relation schema RS:** Set of k attributes $\{A_1,\ldots,A_k\}$
- Attribute A_j is based on **value domain D_j**

My First Table

A1	A2	A3
3	7	T
1	2	T
3	4	F
1	7	T

Tuples

Attribute

HCC Institute of HUMAN-CENTRED COMPUTING

Relational Algebra

- **Relational Algebra**
  - **Operands:** relations (e.g., normalized DB tables)
  - **Operators**: traditional set operations and specific relational operations

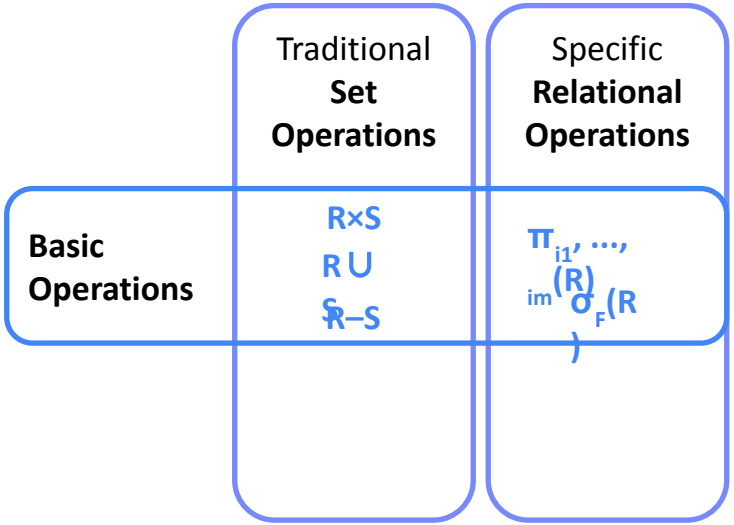
Relational Algebra

- **Relational Algebra**
  - **Operands:** relations (e.g., normalized DB tables)
  - **Operators**: traditional set operations and specific relational operations

  - **Relational operations**:
    - Take one or two relations as input
    - Produce a relation as output
    - Relational operations can be composed together
      - Each operation produces a relation
      - A **query is simply a relational algebra expression** □ produces a relation

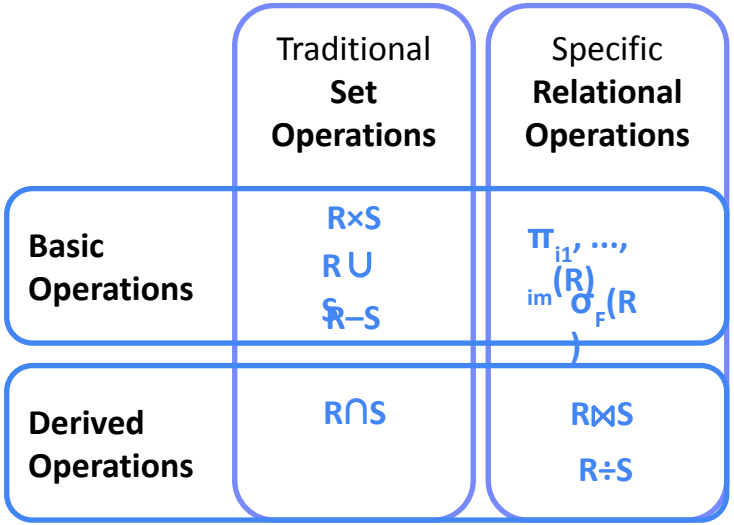
Relational Algebra

- **Basic Operations**
  - Cartesian product $R \times S$
  - Union $R \cup S$
  - Difference $R - S$
  - Projection $\pi_{i1, \ldots, im}(R)$
  - Selection $\sigma_F(R)$



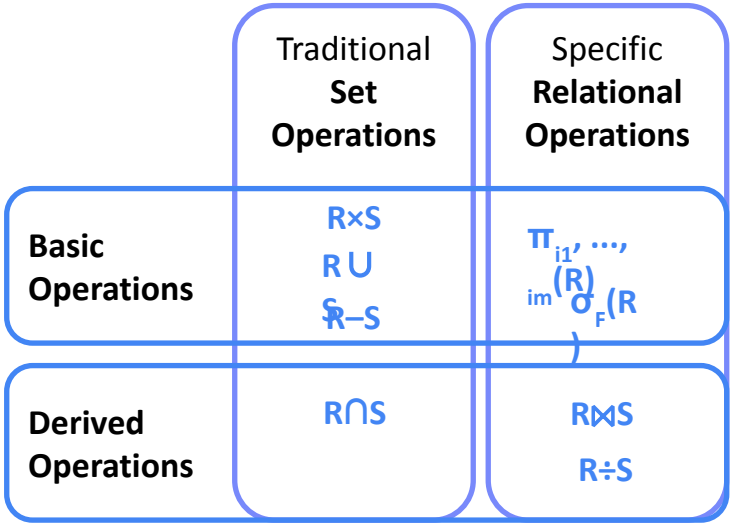
Relational Algebra

- **Basic Operations**
  - Cartesian product $R \times S$
  - Union $R \cup S$
  - Difference $R - S$
  - Projection $\pi_{i1, \ldots, im}(R)$
  - Selection $\sigma_F(R)$
- **Derived Operations**
  - Intersection $R \cap S$
  - Join $R \bowtie S$
  - Division $R \div S$



Relational Algebra

- **Basic Operations**
  - Cartesian product $R \times S$
  - Union $R \cup S$
  - Difference $R - S$
  - Projection $\pi_{i1, \ldots, im}(R)$
  - Selection $\sigma_F(R)$
- **Derived Operations**
  - Intersection $R \cap S$
  - Join $R \bowtie S$
  - Division $R \div S$
- **Extended Operations**



Extended Relational Algebra

- **Set vs. Bag**
  - Relational algebra introduced with **set semantics** (no duplicates) `Set<T>`
  - SQL with **bag semantics** (more flexibility and performance) `Bag<T>`
    - **Multiplicity:** # occurrences of an instance

Extended Relational Algebra

- **Set vs. Bag**
  - Relational algebra introduced with **set semantics** (no duplicates) `Set<T>`
  - SQL with **bag semantics** (more flexibility and performance) `Bag<T>`
    - **Multiplicity:** # occurrences of an instance

- **Additional Operations**
  - Duplicate elimination $\delta(R)$
  - Grouping $\gamma_{A,f(B)}R$
  - Sorting $\tau_A(R)$

Operations

(using DataManagement conference DB + sometimes some other DBs)

PID	Firstname	Lastname	Affiliation	LID
102	Anastasia	Ailamaki	EPFL	1
104	Peter	Bailis	Stanford	
105	Magdalena	Balazinska	U Washington	3
107	Peter	Boncz	CWI	2
108	Surajit	Chaudhuri	MS Research	3
111	Luna	Dong	Amazon	3
113	Juliana	Freire	NYU	5
115	Joe	Hellerstein	UC Berkley	6
116	Stratos	Idreos	Harvard	7
117	Donald	Kossman	MS Research	
118	Tim	Kraska	MIT	7
120	Volker	Markl	TU Berlin	8
122	Tova	Milo	Tel Aviv University	9
123	C.	Mohan	IBM Research	10
124	Thomas	Neumann	TU Munich	11
126	Fatma	Ozcan	IBM Research	10
130	Christopher	Re	Stanford	4

LID	Location
1	Lausanne, SUI
2	Amsterdam, NLD
3	Seattle, USA
4	Stanford, USA
5	New York, USA
6	Berkley, USA
7	Cambridge, USA
8	Berlin, GER
9	Tel Aviv, ISR
10	San Jose, USA
11	Munich, GER

1 to M relationship:
One participant → only one location
A location →more than one participant

TU Graz
HCC Institute of HUMAN-CENTRED COMPUTING

Cartesian Product ✗

- **Definition:** $R \times S := \{(r,s) \mid r \in R, s \in S\}$
  - Set of all pairs of inputs (equivalent in set/bag)

PID	Firstname	Lastname	Affiliation	LID
104	Peter	Bailis	Stanford	
130	Christopher	Re	Stanford	4

LID	Location
4	Stanford, USA
6	Berkley, USA
10	San Jose, USA

Cartesian Product ×

- **Definition:** $R \times S := \{(r,s) \mid r \in R, s \in S\}$
  - Set of all pairs of inputs (equivalent in set/bag)

PID	Firstname	Lastname	Affiliation	LID
104	Peter	Bailis	Stanford	
130	Christopher	Re	Stanford	4

×

LID	Location
4	Stanford, USA
6	Berkley, USA
10	San Jose, USA

↓

PID	Firstname	Lastname	Affiliation	LID	LID	Location
104	Peter	Bailis	Stanford		4	Stanford, USA
130	Christopher	Re	Stanford	4	4	Stanford, USA
104	Peter	Bailis	Stanford		6	Berkley, USA
130	Christopher	Re	Stanford	4	6	Berkley, USA
104	Peter	Bailis	Stanford		10	San Jose, USA
130	Christopher	Re	Stanford	4	10	San Jose, USA

Union ∪

- **Definition:** $R \cup S := \{x \mid x \in R \vee x \in S\}$
  - **Set:** set union with duplicate elimination
  - **Bag:** bag union

Firstname	Lastname	Affiliation
Anastasia	Ailamaki	EPFL
Peter	Boncz	CWI
Volker	Markl	TU Berlin
Thomas	Neumann	TU Munich

Firstname	Lastname	Affiliation
Anastasia	Ailamaki	EPFL
Magdalena	Balazinska	U Washington
Juliana	Freire	NYU
Tova	Milo	Tel Aviv University

Union requires **compatible schema**

Note: Union (∪) has OR semantics (∨)

∪

Firstname	Lastname	Affiliation
Anastasia	Ailamaki	EPFL
Peter	Boncz	CWI
Volker	Markl	TU Berlin
Thomas	Neumann	TU Munich
Magdalena	Balazinska	U Washington
Juliana	Freire	NYU
Tova	Milo	Tel Aviv University

Difference –

- **Definition: R–S := {x | x ∈ R ⋀ x ∉ S}** (sometimes \)
  - **Set:** set difference
  - **Bag:** Each tuple appears the number of times it appears in R, minus # times appears in S, but never less than 0.

Affiliation
Stanford
U Washington
MS Research
Amazon
NYU
UC Berkley
Harvard
MIT
IBM Research

–

Affiliation
Stanford
U Washington
NYU
UC Berkley
Harvard
MIT
TU Berlin
Stanford

Difference requires **compatible schema**

Affiliation
MS Research
Amazon
IBM Research

Projection $\pi_{i1,\ldots,im}(R)[pi]$

- **Set:** selection of attributes with duplicate elimination
- **Bag:** selection of attributes
- Result relation contains only the specified attributes of **R**

Example:

$\pi_{Affiliation}(Participant)$
w/ set semantics

Affiliation
EPFL
Stanford
U Washington
CWI
MS Research
Amazon
NYU
UC Berkley
Harvard
MIT
TU Berlin
Tel Aviv University
IBM Research
TU Munich

Projection

Selection of multiple attributes

CID	Title	ECTS	PID
INF.01017UF	Data Management (VO)	3	7
706.010	Databases	3	18

$$\Pi_{\text{Title,ECTS}}(\text{Lectures})$$

Title	ECTS
Data Management (VO)	3
Databases	3

Extended Projection

- Arithmetic expressions: $\pi_{A*B}(R)$

$\pi_{ECTS*Score}(Lectures)$

CID	Title	ECTS	Score
INF.01017UF	Data Management (VO)	3	2
706.010	Databases	3	8

ECTS * Score
6
24

Selection $\sigma_F(R)$ [sigma]

- Selection of tuples satisfying the predicate F (equivalent in set/bag)
- **F** is the predicate for selection
  - F can refer to attributes in R
  - Can use comparison operators: =, ≠, <, ≤, >, ≥
  - Can combine multiple predicates using: ∧ (and), ∨ (or), ¬ (not)
- **R** is the input relation
- Result relation contains all tuples in **R** for which **F** is true
- Result schema is identical to schema for **R**

TU Graz
HCC Institute of HUMAN-CENTRED COMPUTING

Selection $\sigma_F(R)$

$$\sigma_{\text{Affiliation='IBM Research'}}(\text{Participant})$$

PID	Firstname	Lastname	Affiliation	LID
123	C.	Mohan	IBM Research	10
126	Fatma	Ozcan	IBM Research	10

Questions? Comments?

Derived Operations

Remember: Relational Algebra

- **Basic Operations** (minimal)
  - Cartesian product $R \times S$
  - Union $R \cup S$
  - Difference $R - S$
  - Projection $\pi_{i1, \ldots, im}(R)$
  - Selection $\sigma_F(R)$
- **Derived Operations**
  - Intersection $R \cap S$
  - Join $R \bowtie S$
  - Division $R \div S$
- **Extended Operations**

	Traditional **Set Operations**	Specific **Relational Operations**
Basic Operations	$R \times S$ $R \cup S$ $R - S$	$\pi_{i1, \ldots, im}(R)$ $\sigma_F(R)$
Derived Operations	$R \cap S$	$R \bowtie S$ $R \div S$

Intersection ∩

- **Definition R∩S := R – (R – S)** (derived from basic operations)
  - **Set:** set intersection derived from difference
  - **Bag:** bag intersection, with element multiplicity min(R,S)

Firstname	Lastname	Affiliation
Anastasia	**Ailamaki**	**EPFL**
Peter	Boncz	CWI
Volker	Markl	TU Berlin
Thomas	Neumann	TU Munich

Firstname	Lastname	Affiliation
Anastasia	**Ailamaki**	**EPFL**
Magdalena	Balazinska	U Washington
Juliana	Freire	NYU
Tova	Milo	Tel Aviv University

∩

Firstname	Lastname	Affiliation
Anastasia	Ailamaki	EPFL

Note: Intersection (∩) has **AND semantics** (∧)

Intersection requires **compatible schema**

(Inner) Join

- **Definition $R \bowtie S := \pi\ldots (\sigma_F (R \times S))$**
  - Selection of tuples (and attributes) from the cartesian product R×S (equivalent in set/bag); **beware of NULLs:** do never match



(Inner) Join

- Definition $R \bowtie S := \pi\ldots (\sigma_F (R \times S))$
  - Selection of tuples (and attributes) from the cartesian product R×S (equivalent in **set**/**bag**); **beware of NULLs:** do never match
  - **Theta Join:** $R \bowtie_\Theta S := \sigma_\Theta (R \times S)$; arbitrary condition e.g., $\Theta \in \{=, \neq, <, \leq, >, \geq\}$

Sailor

SID	SName	Age
22	Dustin	45
31	Lubber	55
58	Rusty	35

Boats

BID	BName	color
101	Blue moon	blue
103	Vitamin Sea	white
105	Boita	green

Reserved

SID	BID	Day
22	101	10/10/2023
58	103	11/12/2023

- **Theta Join: Sailor $\bowtie_{S.sid<R.sid}$ Reserved**

SID	SName	Age	SID	BID	Day
22	Dustin	45	58	103	11/12/2023
31	Lubber	55	58	103	11/12/2023

Sailor

SID	SName	Age
22	Dustin	45
31	Lubber	55
58	Rusty	35

Boats

BID	BName	color
101	Blue moon	blue
103	Vitamin Sea	white
105	Boita	green

Reserved

SID	BID	Day
22	101	10/10/2023
58	103	11/12/2023

- **Theta Join: Sailor $\bowtie_{S.sid<R.sid}$ Reserved**

SID	SName	Age	SID	BID	Day
22	Dustin	45	58	103	11/12/2023
31	Lubber	55	58	103	11/12/2023

Join in operators different than = are important when you do not have identically matching columns (e.g. timestamps)

(Inner) Join

- **Definition $R \bowtie S := \pi\ldots (\sigma_F (R \times S))$**
  - Selection of tuples (and attributes) from the cartesian product R×S (equivalent in **set**/**bag**); **beware of NULLs:** do never match
  - **Theta Join:** $R \bowtie_\Theta S := \sigma_\Theta (R \times S)$; arbitrary condition e.g., $\Theta \in \{=, \neq, <, \leq, >, \geq\}$
  - **Equi join:** (Θ is =)

Sailor

SID	SName	Age
22	Dustin	45
31	Lubber	55
58	Rusty	35

Boats

BID	BName	color
101	Blue moon	blue
103	Vitamin Sea	white
105	Boita	green

Reserved

SID	BID	Day
22	101	10/10/2023
58	103	11/12/2023

- **Equi Join: Sailor $\bowtie_{R.sid=S.sid}$ Reserved**

SID	SName	Age	SID	BID	Day
22	Dustin	45	22	101	10/10/2023
58	Rusty	35	58	103	11/12/2023

Sailor

SID	SName	Age
22	Dustin	45
31	Lubber	55
58	Rusty	35

Boats

BID	BName	color
101	Blue moon	blue
103	Vitamin Sea	white
105	Boita	green

Reserved

SID	BID	Day
22	101	10/10/2023
58	103	11/12/2023

- **Equi Join: Sailor $\bowtie_{R.sid=S.sid}$ Reserved**

SID	SName	Age	SID	BID	Day
22	Dustin	45	22	101	10/10/2023
58	Rusty	35	58	103	11/12/2023

Still we have redundant attributes

(Inner) Join

- Definition $R \bowtie S := \pi\ldots (\sigma_F (R \times S))$
  - Selection of tuples (and attributes) from the cartesian product R×S (equivalent in **set**/**bag**); **beware of NULLs:** do never match
  - **Theta Join:** $R \bowtie_\Theta S := \sigma_\Theta (R \times S)$; arbitrary condition e.g., $\Theta \in \{=, \neq, <, \leq, >, \geq\}$
  - **Equi join:** (Θ is =)
  - **Natural Join:** $R \bowtie S$ w/ shared attributes appearing once

Sailor

SID	SName	Age
22	Dustin	45
31	Lubber	55
58	Rusty	35

Boats

BID	BName	color
101	Blue moon	blue
103	Vitamin Sea	white
105	Boita	green

Reserved

SID	BID	Day
22	101	10/10/2023
58	103	11/12/2023

- **Natural Join Sailor ⋈ Reserved**

SID	SName	Age	BID	Day
22	Dustin	45	101	10/10/2023
58	Rusty	35	103	11/12/2023

Another Example

- **Natural Join Participant ⋈ Location**

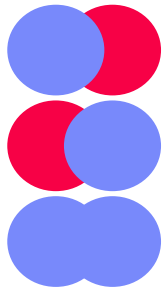
PID	Firstname	Lastname	Affiliation	LID
102	Anastasia	Ailamaki	EPFL	1
104	Peter	Bailis	Stanford	
105	Magdalena	Balazinska	U Washington	3



PID	Firstname	Lastname	Affiliation	LID	Location
102	Anastasia	Ailamaki	EPFL	1	Lausanne, SUI
105	Magdalena	Balazinska	U Washington	3	Seattle, USA

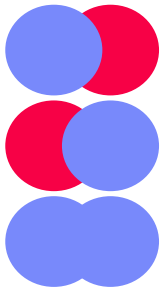
Other Types of Joins

- **Outer Joins**
  - Left outer join ⟕ (tuples of lhs, NULLs for non-existing rhs)
  - Right outer join ⟖ (tuples of rhs, NULLs for non-existing lhs)
  - Full outer join ⟗ (tuples of lhs/rhs, NULLs for non-existing lhs/rhs)



Other Types of Joins

- **Outer Joins**
  - **Left outer join ⟕ (tuples of lhs, NULLs for non-existing rhs)**
  - Right outer join ⟖ (tuples of rhs, NULLs for non-existing lhs)
  - Full outer join ⟗ (tuples of lhs/rhs, NULLs for non-existing lhs/rhs)



Participant ⟕ Location

PID	Firstname	Lastname	Affiliation	LID	LID	Location
102	Anastasia	Ailamaki	EPFL	1	1	Lausanne, SUI
104	Peter	Bailis	Stanford	NULL	NULL	NULL
105	Magdalena	Balazinska	UW	3	3	Seattle, USA

Other Types of Joins

- **Semi Join**
  - Left semi join $\ltimes := \pi_R(R \bowtie S)$ (filter lhs)
  - Right semi join $\rtimes$ (filter rhs)
  - If we only want to have attributes from one table

Other Types of Joins

- **Semi Join**
  - Left semi join $\ltimes := \pi_R(R \bowtie S)$ (filter lhs)
  - Right semi join $\rtimes$ (filter rhs)
  - If we only want to have attributes from one table

$$\text{Participant} \ltimes \sigma_{\text{Location LIKE \%GER\%}}(\text{Location})$$

PID	Firstname	Lastname	Affiliation	LID
120	Volker	Markl	TU Berlin	8
124	Thomas	Neumann	TU Munich	11

Division

- **Definition R÷S**
  - Find all tuples in R that have one tuple corresponding to each tuple in S
  - Find instances in R that satisfy S

  - **Allows to ask questions like:**
    - **Which students took ALL DB-related courses?**

Questions / Comments?

Extended Operations

Deduplication, Sorting, and Renaming

- **Duplication Elimination δ(R) [delta]**
  - Convert a **bag** into a **set** by removing all duplicate instances
  - SQL: use **ALL** or **DISTINCT** to indicate w/ or w/o duplicate elimination

Deduplication, Sorting, and Renaming

- **Duplication Elimination δ(R) [delta]**
  - Convert a **bag** into a **set** by removing all duplicate instances
  - SQL: use **ALL** or **DISTINCT** to indicate w/ or w/o duplicate elimination

- **Sorting $\tau_A(R)$ [tau]**
  - Convert a **bag** into a **sorted list** of tuples; order lost if used in other ops
  - SQL: sequence of attributes with ASC (ascending) or DESC (descending) order
  - Example: $\tau_{\text{Firstname ASC, Lastname DESC}}(\text{Participant})$

Deduplication, Sorting, and Renaming

- **Rename $\rho_s(R)$ [rho]**
  - Define new schema (attribute names), but keep tuples unchanged
  - Like aliases in SQL
  - **Example:** $\rho_{\text{ID, Given Name, Family Name, Affiliation, LID}}(\text{Participant})$

Grouping and Aggregation $\gamma_{A,f(B)}R$ [gamma]

- **Grouping:** group input tuples R according to unique values in A
- **Aggregation:** compute aggregate f(B) per group of tuples

- **Example** $\gamma_{Affiliation,COUNT(*)}Participant$

Affiliation	COUNT
EPFL	1
Stanford	2
U Washington	1
CWI	1
MS Research	2
Amazon	1
...	...
IBM Research	2
TU Munich	1

Grouping and Aggregation $\gamma_{A,f(B)}R$ [gamma]

- **Grouping:** group input tuples R according to unique values in A
- **Aggregation:** compute aggregate f(B) per group of tuples

- **Example** $\gamma_{Affiliation,COUNT(*)}Participant$

- **SUM, COUNT, MIN, MAX, AVG, MEDIAN**
- **If you specify a column name instead of (*)**

NULL values will not be counted

Affiliation	COUNT
EPFL	1
Stanford	2
U Washington	1
CWI	1
MS Research	2
Amazon	1
...	...
IBM Research	2
TU Munich	1

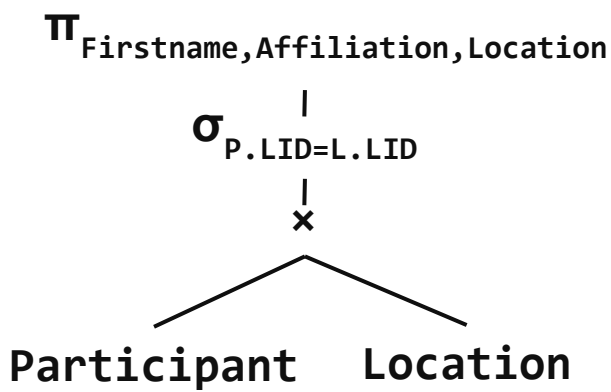
Logical Query Trees Query Processing

TU Graz
HCC Institute of HUMAN-CENTRED COMPUTING

Logical Query Trees Query Processing

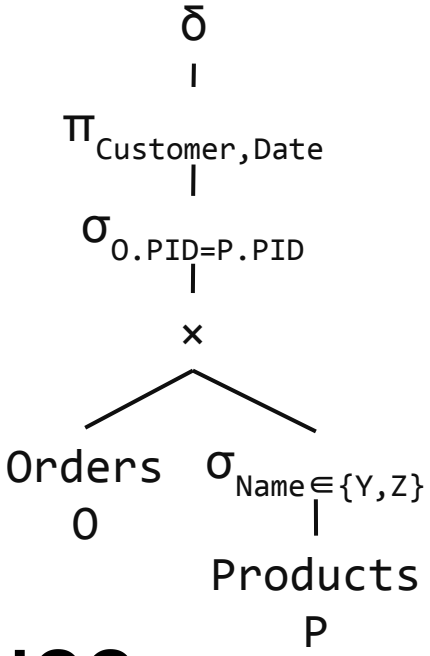
- **Composition of Complex Queries**
  - Relational algebra expressions data flow graph (tree)
  - Leaf nodes represent base relations; root node represents result

$$\pi_{\texttt{Firstname,Affiliation,Location}}\ (\sigma_{\texttt{P.LID=L.LID}}(\texttt{Participant} \times \texttt{Location}))$$



Firstname	Affiliation	Location
Anastasia	EPFL	Lausanne, SUI
Magdalena	U Washington	Seattle, USA
Peter	CWI	Amsterdam, NLD
Surajit	MS Research	Seattle, USA
...	...	...

Practice: Compute the results for these queries

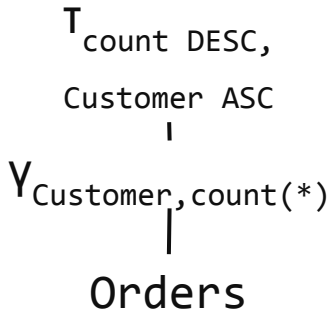


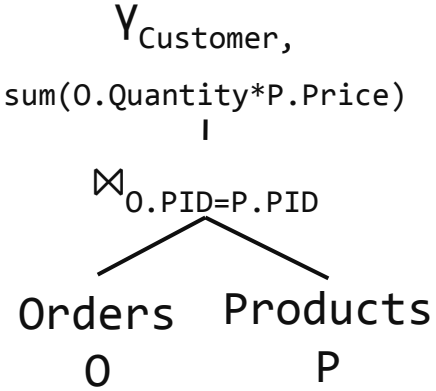
Orders

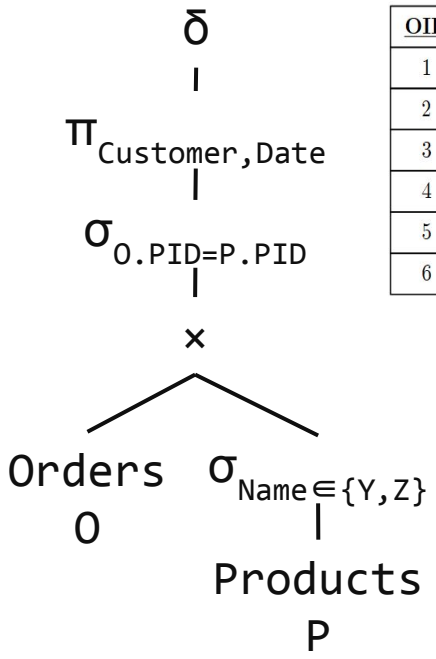
OID	Customer	Date	Quantity	PID
1	A	'2019-06-22'	3	2
2	B	'2019-06-22'	1	3
3	A	'2019-06-22'	1	4
4	C	'2019-06-23'	2	2
5	D	'2019-06-23'	1	4
6	C	'2019-06-23'	1	1

Products

PID	Name	Price
1	X	100
2	Y	15
4	Z	75
3	W	120







Orders

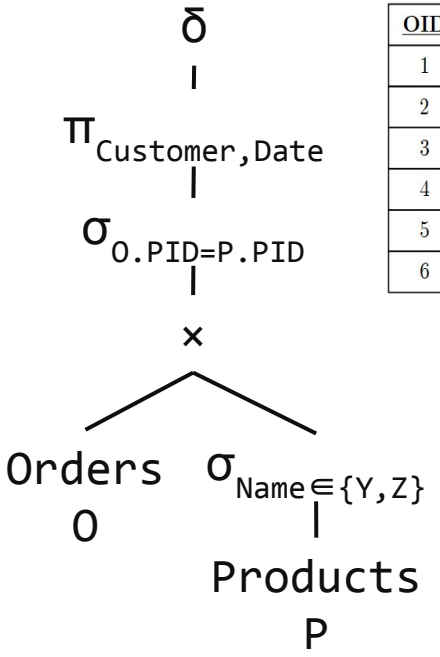
OID	Customer	Date	Quantity	PID
1	A	'2019-06-22'	3	2
2	B	'2019-06-22'	1	3
3	A	'2019-06-22'	1	4
4	C	'2019-06-23'	2	2
5	D	'2019-06-23'	1	4
6	C	'2019-06-23'	1	1

Products

PID	Name	Price
1	X	100
2	Y	15
4	Z	75
3	W	120

Customer	Date
A	'2019-06-22'
C	'2019-06-23'
D	'2019-06-23'

TU Graz
HCC Institute of HUMAN-CENTRED COMPUTING



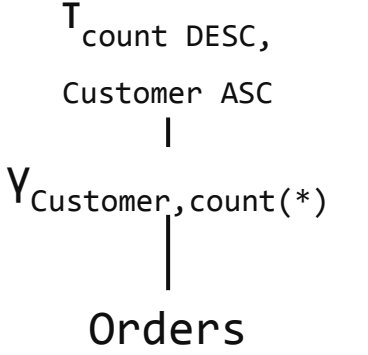
Orders

OID	Customer	Date	Quantity	PID
1	A	'2019-06-22'	3	2
2	B	'2019-06-22'	1	3
3	A	'2019-06-22'	1	4
4	C	'2019-06-23'	2	2
5	D	'2019-06-23'	1	4
6	C	'2019-06-23'	1	1

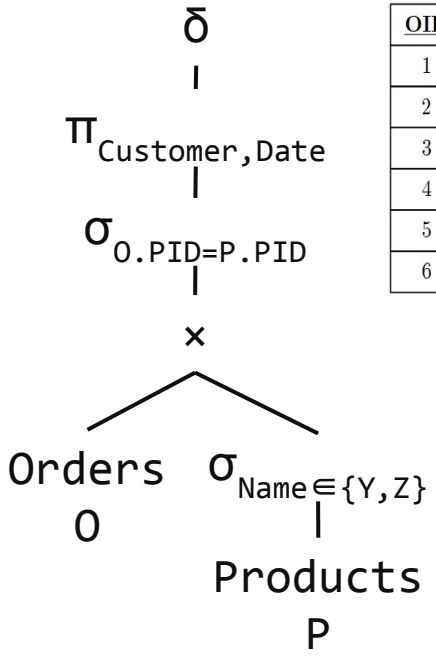
Products

PID	Name	Price
1	X	100
2	Y	15
4	Z	75
3	W	120

Customer	Date
A	'2019-06-22'
C	'2019-06-23'
D	'2019-06-23'



Customer	Count
A	2
C	2
B	1
D	1



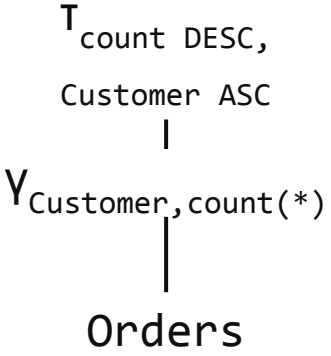
Orders

OID	Customer	Date	Quantity	PID
1	A	'2019-06-22'	3	2
2	B	'2019-06-22'	1	3
3	A	'2019-06-22'	1	4
4	C	'2019-06-23'	2	2
5	D	'2019-06-23'	1	4
6	C	'2019-06-23'	1	1

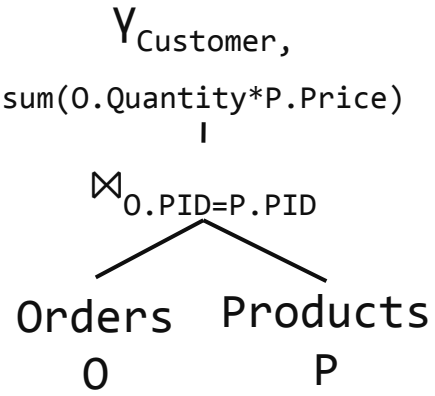
Products

PID	Name	Price
1	X	100
2	Y	15
4	Z	75
3	W	120

Customer	Date
A	'2019-06-22'
C	'2019-06-23'
D	'2019-06-23'



Customer	Count
A	2
C	2
B	1
D	1



Customer	Sum
A	120
B	120
C	130
D	75

Query Rewriting and Optimization

= the process of replacing a non-optimal query with a more efficient one that provides an equivalent output

Standardization and Simplification

- **Transformation Rules for Boolean Expressions**

Rule Name	Examples
Commutativity rules	A OR B ⇔ B OR A A AND B ⇔ B AND A
Associativity rules	(A OR B) OR C ⇔ A OR (B OR C) (A AND B) AND C ⇔ A AND (B AND C)
Distributivity rules	A OR (B AND C) ⇔ (A OR B) AND (A OR C) A AND (B OR C) ⇔ (A AND B) OR (A AND C)
De Morgan's rules	NOT (A AND B) ⇔ NOT (A) OR NOT (B) NOT (A OR B) ⇔ NOT (A) AND NOT (B)
Double-negation rules	NOT(NOT(A)) ⇔ A
Idempotence rules	A OR A ⇔ A A AND A ⇔ A A OR NOT(A) ⇔ TRUE A AND NOT (A) ⇔ FALSE A AND (A OR B) ⇔ A A OR (A AND B) ⇔ A A OR FALSE ⇔ A A AND TRUE ⇔ A A AND FALSE ⇔ FALSE A OR TRUE ⇔ TRUE

Standardization and Simplification

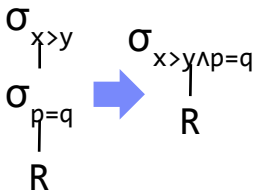
- **Elimination of Common Subexpressions (redundancy)**
  - $(A_1=a_{11}$ **OR** $A_1=a_{12})$ **AND** $(A_1=a_{12}$ **OR** $A_1=a_{11})$ □ $A_1=a_{11}$ **OR** $A_1=a_{12}$
- **Propagation of Constants**
  - A ≥ **B** **AND** B = **7** □ A ≥ **7** **AND** B = **7**
- **Detection of Contradictions (e.g., cycles)**
  - A ≥ B **AND** B > C **AND** C ≥ A □ **A > A** ⟶ **FALSE**

Standardization and Simplification

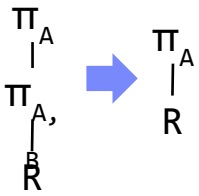
- **Use of Constraints**
  - A is primary key/unique → **no duplicate elimination necessary**
  - `Rule STATUS = ‘married’ -> TAX_CLASS ≥ 3:`
    `(STATUS = ‘married’ AND TAX_CLASS = 1)  FALSE`
- **Elimination of Redundancy:**
  - $R \bowtie R \rightarrow R$
  - $R \cup R \rightarrow R$
  - $R - R \rightarrow \emptyset$
  - $R \bowtie (\sigma_p R) \rightarrow \sigma_p R$
  - $R \cup (\sigma_p R) \rightarrow R$
  - $(\sigma_{p1} R) \bowtie (\sigma_{p2} R) \rightarrow \sigma_{p1 \wedge p2} R$
  - $(\sigma_{p1} R) \cup (\sigma_{p2} R) \rightarrow \sigma_{p1 \vee p2} R$

Selections and Projections – Transformation Rules

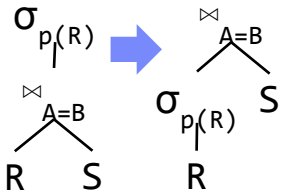
1) Grouping of Selections



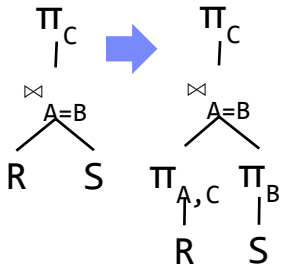
2) Grouping of Projections



3) Pushdown of Selections



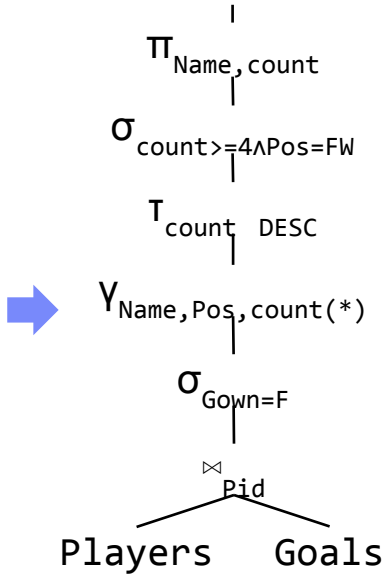
4) Pushdown of Projections



Practice (at home)

Example:
Football database

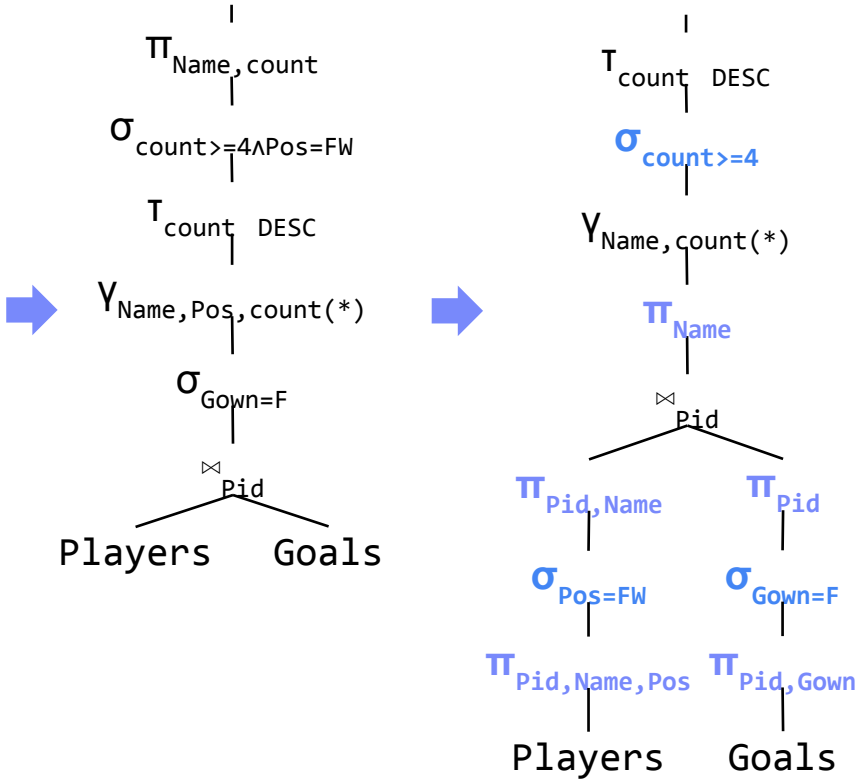
Query:
Which player (name) scored at least 4 non-own goals in the position foward?



Practice (at home)

Example:
Football database

Query:
Which player (name) scored at least 4 non-own goals in the position foward?



Practice (at home)

- **Rewrite the following RA expressions – assuming two relations R(a, b, c) and S(d, e, f) – into equivalent expressions with lower costs.**
  - $\sigma_{b=7}(R \bowtie S)$
  - $(\sigma_{e>3}(S)) \cap (\sigma_{f<7}(S))$
  - $R \cup (\sigma_{d<e \wedge e<f \wedge f<d}(S))$
  - $\sigma_{b=3}(\gamma_{b,max(c)}(R))$

Practice (at home)

- **Rewrite the following RA expressions – assuming two relations R(a, b, c) and S(d, e, f) – into equivalent expressions with lower costs.**

  - $\sigma_{b=7}(R \bowtie S)$
  - $(\sigma_{e>3}(S)) \cap (\sigma_{f<7}(S))$
  - $R \cup (\sigma_{d<e \wedge e<f \wedge f<d}(S))$
  - $\sigma_{b=3}(\gamma_{b,max(c)}(R))$

  - $\sigma_{b=7}(R) \bowtie S$
  - $\sigma_{e>3 \wedge f<7}(S)$
  - $R \cup \emptyset \equiv R$
  - $\gamma_{b,max(c)}(\sigma_{b=3}(R))$

Practice (at home)

- **Assume relations R(a,b,c) and S(d,e), and indicate in the table below whether or not the two RA expressions per row are equivalent in bag semantics. For non-equivalent expressions briefly explain why.**

Expression 1	Expression 2	Equivalent?
$\sigma_{c=3}(\sigma_{b=7}(R))$	$\sigma_{c=3}(\sigma_{c=3 \vee b=7}(R))$	✗
$R \bowtie_{a=e} S$	$\sigma_{a=e}(R \times S)$	✓
$(\sigma_{b<3}(R)) \cap (\sigma_{b \geq 3}(R))$	R	✗
$\pi_{b,d}(R \bowtie_{a=e} S)$	$(\pi_{a,b}(R)) \bowtie_{a=e} (\pi_{d,e}(S))$	✗
$\pi_{a,b}(\sigma_{c=3}(\sigma_{b=7}(R)))$	$\sigma_{b=7}(\pi_{a,b}(\sigma_{c=3}(R)))$	✓

- **1) b=7 not enforced, 3) union would be needed, 4) only b, d is asked**

Other Optimization Methods: Query Unnesting

- **Query unnesting**
  - Get rid of expensive and unnecessary sub-queries
  - Multiple variants possible
    - E.g., sometimes a subquery can be substituted by a simple join
  - One example:

[Won Kim: On Optimizing an SQL-like Nested Query. **ACM Trans. Database Syst. 1982**]



```
SELECT OrderID FROM Order
  WHERE ProdID IN
   (SELECT ProdID
      FROM Product WHERE Price<100)
```

```
SELECT OrderNo
 FROM Order O, Product P
 WHERE O.ProdID = P.ProdID
   AND P.Price < 100
```

Physical Operators & Query Processing

TU Graz
HCC Institute of HUMAN-CENTRED COMPUTING

Volcano Iterator Model

[Goetz Graefe: Volcano - An Extensible and Parallel Query Evaluation System. **IEEE Trans. Knowl. Data Eng. 1994**]



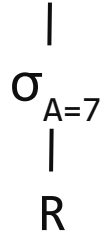
- **Pipelined** → one iterator calls the next one with its input
- **Open-Next-Close** (ONC) interface (implemented by every operator)
- **Query plans** list the physical operators needed to execute a query
  - They have **costs** → the „planner“ chooses the plan with the smallest costs

Example $\sigma_{A=7}(R)$ → seq scan / table scan

```
void open() { R.open(); }

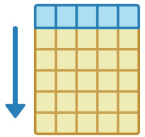
void close() { R.close(); }

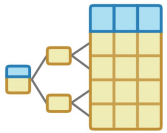
Record next() {
  while( (r = R.next()) != EOF )
    if( p(r) ) //A==7
      return r;
  return EOF;
}
```

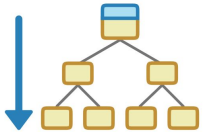


(Selected) Physical Operators in PostgreSQL

- Physical Table Access Operators (σ)
  - **Seq Scan (table scan)**
    - Go over tuple by tuple







  - **Index Scan**
    - Use index (like a virtual table based on data from other tables – details in next lecture) to speed up process

  - **Index Only Scan**
    - If you do not need access to specific attributes even faster

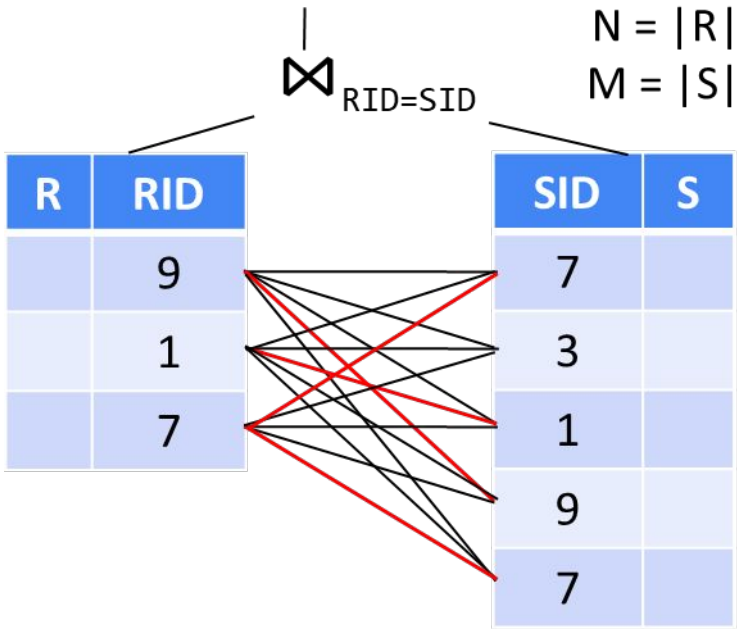
[icons taken from the PgAdmin Visual Explain Tool]

[**Images:** PostgreSQL/11/pgAdmin 4/web/pgadmin/misc/static/explain/img]

(Selected) Physical Operators in PostgreSQL

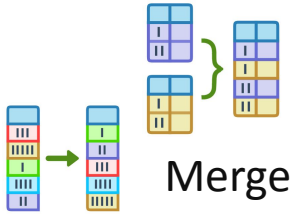
- Physical Join Operators (⋈)
  - **Nested Loop Join**
    - Most general join
    - Works for all join conditions
    - Like two nested loops
      - For every element in R □
        search element in S □
        If element is found □ join
  - **Complexity**
    - Time: O(N * M)
    - Space: O(1)
    - Very slow but constant memory consumption



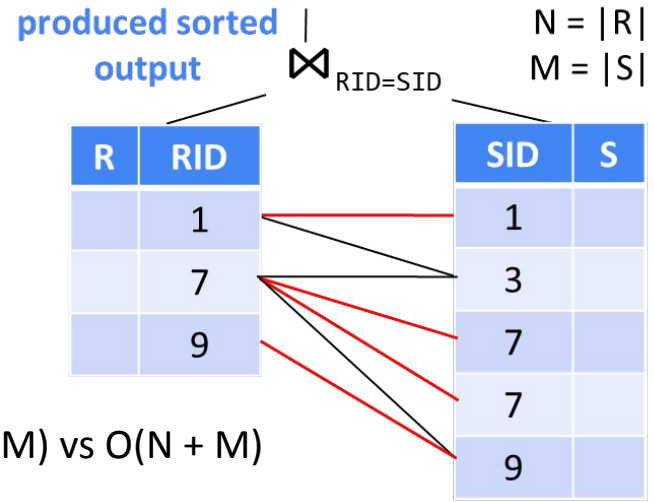


[**Images:** PostgreSQL/11/pgAdmin 4/web/pgadmin/misc/static/explain/img]

(Selected) Physical Operators in PostgreSQL

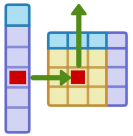


- Physical Join Operators (⋈)
  - **Sort Merge Join**
    - Sort phase (with some sorting algorithm)
      - Sort input tables R and S
    - Merge phase
      - Make use of the sorted indices
      - If RID < SID □ look at next RID
      - If RID > SID □ look at next SID
      - If RIF == SID □ join
  - **Complexity**
    - Time (unsorted vs sorted): O(N log N + M log M) vs O(N + M)
    - Space (unsorted vs sorted): O(N + M) vs O(1)



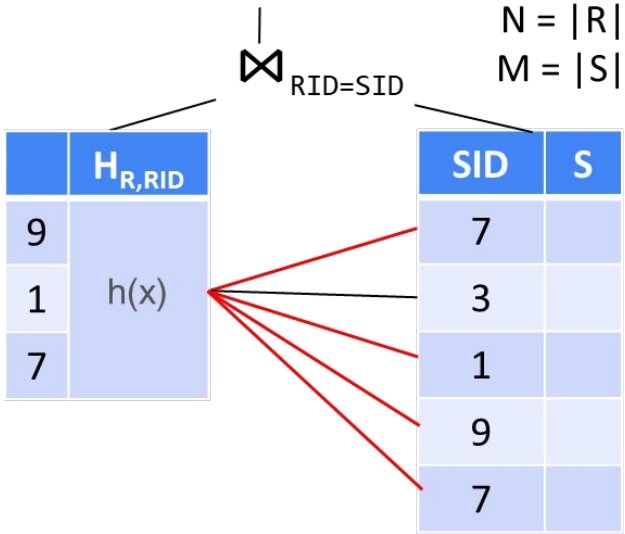
[**Images:** PostgreSQL/11/pgAdmin 4/web/pgadmin/misc/static/explain/img]

(Selected) Physical Operators in PostgreSQL



Hash

- Physical Join Operators (⋈)
  - **Hash Join**
    - Build phase (with some hash function to map keys to fixed-sized set of values)
      - Read table R and build a hash table H_R over the join key
    - Probe phase
      - Read table S and probe H_R with the join key
      - If key is found □ join
  - **Complexity**
    - Time: O(N + M)
    - Space: O(N)



[**Images:** PostgreSQL/11/pgAdmin 4/web/pgadmin/misc/static/explain/img]

SETTING Analyze + Explain
□ get query plan & costs

Participants

PID	...	Affiliation	LID
102		EPFL	1
104		Stanford	
105		U Washington	3
107		CWI	2
108		MS Research	3
111		Amazon	3
113		NYU	5
115		UC Berkley	6
116		Harvard	7
117		MS Research	
118		MIT	7
120		TU Berlin	8
122		Tel Aviv University	9
123		IBM Research	10
124		TU Munich	11
126		IBM Research	10
130		Stanford	4

Location

LID	Location
1	Lausanne, SUI
2	Amsterdam, NLD
3	Seattle, USA
4	Stanford, USA
5	New York, USA
6	Berkley, USA
7	Cambridge, USA
8	Berlin, GER
9	Tel Aviv, ISR
10	San Jose, USA
11	Munich, GER

ANALYZE and EXPLAIN

- **Step 1:** `EXPLAIN SELECT * FROM Participant AS R, Location AS S WHERE R.LID=S.LID;`

```
Hash Join (.. rows=70 width=1592)
  Hash Cond:(s.lid = r.lid)
  ->  Seq Scan on location s (.. rows=140 width=520)
  ->  Hash (.. rows=70 width=1072)
        ->  Seq Scan on participant r (.. rows=70 width=1072)
```

ANALYZE and EXPLAIN

- **Step 1:** `EXPLAIN SELECT * FROM Participant AS R, Location AS S WHERE R.LID=S.LID;`

```
Hash Join (.. rows=70 width=1592)
  Hash Cond:(s.lid = r.lid)
  ->  Seq Scan on location s (.. rows=140 width=520)
  ->  Hash (.. rows=70 width=1072)
        ->  Seq Scan on participant r (.. rows=70 width=1072)
```

- **Step 2:** `ANALYZE Participant, Location;`

ANALYZE and EXPLAIN

- **Step 1:** `EXPLAIN SELECT * FROM Participant AS R, Location AS S WHERE R.LID=S.LID;`

```
Hash Join (.. rows=70 width=1592)
  Hash Cond:(s.lid = r.lid)
  ->  Seq Scan on location s (.. rows=140 width=520)
  ->  Hash (.. rows=70 width=1072)
        ->  Seq Scan on participant r (.. rows=70 width=1072)
```

- **Step 2:** `ANALYZE Participant, Location;`

- **Step 3:** `EXPLAIN SELECT * FROM Participant AS R, Location AS S WHERE R.LID=S.LID;`

```
Hash Join (.. rows=17 width=47)
  Hash Cond:(r.lid = s.lid)
  ->  Seq Scan on participant r (.. rows=17 width=30)
  ->  Hash (.. rows=11 width=17)
        ->  Seq Scan on location s (.. rows=11 width=17)
```

ANALYZE and EXPLAIN

- **Step 1:** `EXPLAIN SELECT * FROM Participant AS R, Location AS S WHERE R.LID=S.LID;`

```
Hash Join (.. rows=70 width=1592)
  Hash Cond:(s.lid = r.lid)
  ->  Seq Scan on location s (.. rows=140 width=520)
  ->  Hash (.. rows=70 width=1072)
        ->  Seq Scan on participant r (.. rows=70 width=1072)
```

- **Step 2:** `ANALYZE Participant, Location;`

- **Step 3:** `EXPLAIN SELECT * FROM Participant AS R, Location AS S WHERE R.LID=S.LID;`

```
Hash Join (.. rows=17 width=47)
  Hash Cond:(r.lid = s.lid)
  ->  Seq Scan on participant r (.. rows=17 width=30)
  ->  Hash (.. rows=11 width=17)
        ->  Seq Scan on location s (.. rows=11 width=17)
```

Syntax:
EXPLAIN (ANALYZE ON, FORMAT JSON)

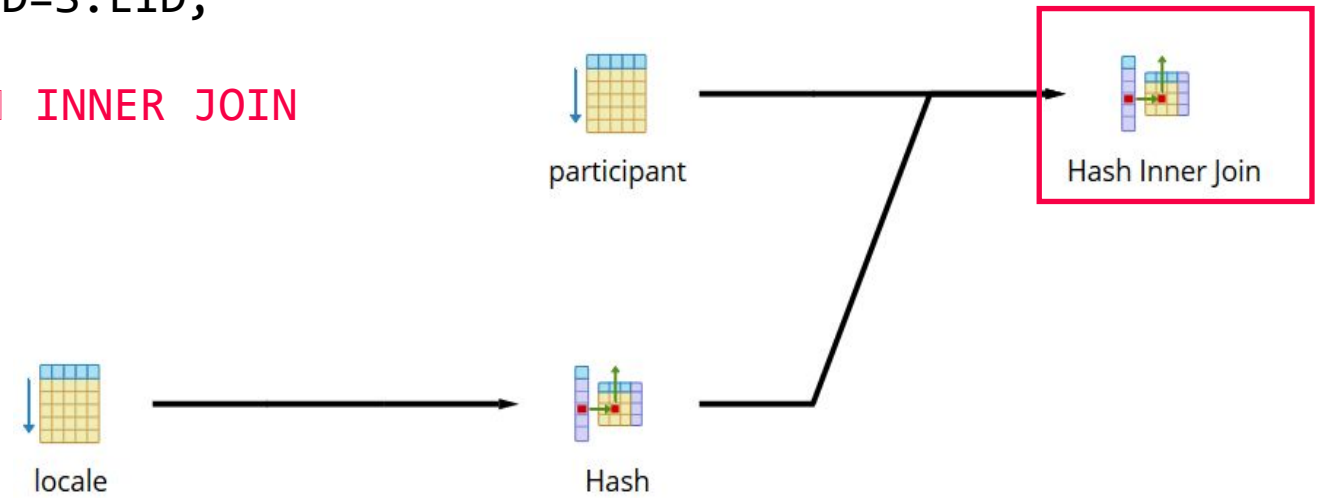
- Query plan including costs
- **Important are the costs in the most outer element**

Visual EXPLAIN

- In PgAdmin □ query tool □ Explain tab

- ```
  SELECT * FROM Participant AS R, Locale AS S
    WHERE R.LID=S.LID;
  ```

$\sigma_F(R \times S)$ □ INNER JOIN


  ```

Conclusions

- **Summary**
  - **Relational algebra**
  - **Query optimization including Analyze+Explain**

Conclusions

- **Summary**
  - Relational algebra
  - Query optimization including Analyze+Explain

- **Next week (27.04.2026)**
  - Physical design and transaction processing
- **Questions / comments?**
  - Discord (for KU)